T.Chaitra (192425217)

Experiment 1: AI Maze Escape

Title

AI Maze Escape – Finding the Shortest Path

Aim

To find the shortest path from the START position to the GOAL position in a maze while avoiding obstacles using an AI search strategy.

Problem Statement

An AI robot is placed inside a maze and must navigate from the START point to the GOAL point. The objective is to reach the destination by following the shortest possible path without crossing any walls.

Algorithm Used

Breadth-First Search (BFS)

Theory

Breadth-First Search (BFS) is an uninformed search algorithm that explores all neighboring nodes level by level. It guarantees finding the shortest path in an unweighted maze by visiting each possible position systematically.

Implementation

1. Open the Maze Escape game in Scratch.
2. Click the **Green Flag** to start the game.
3. Use the arrow keys to move the player through the maze.
4. Avoid touching the maze walls.
5. Continue moving until the goal is reached.
6. The shortest valid path is recorded after reaching the destination.

Observation

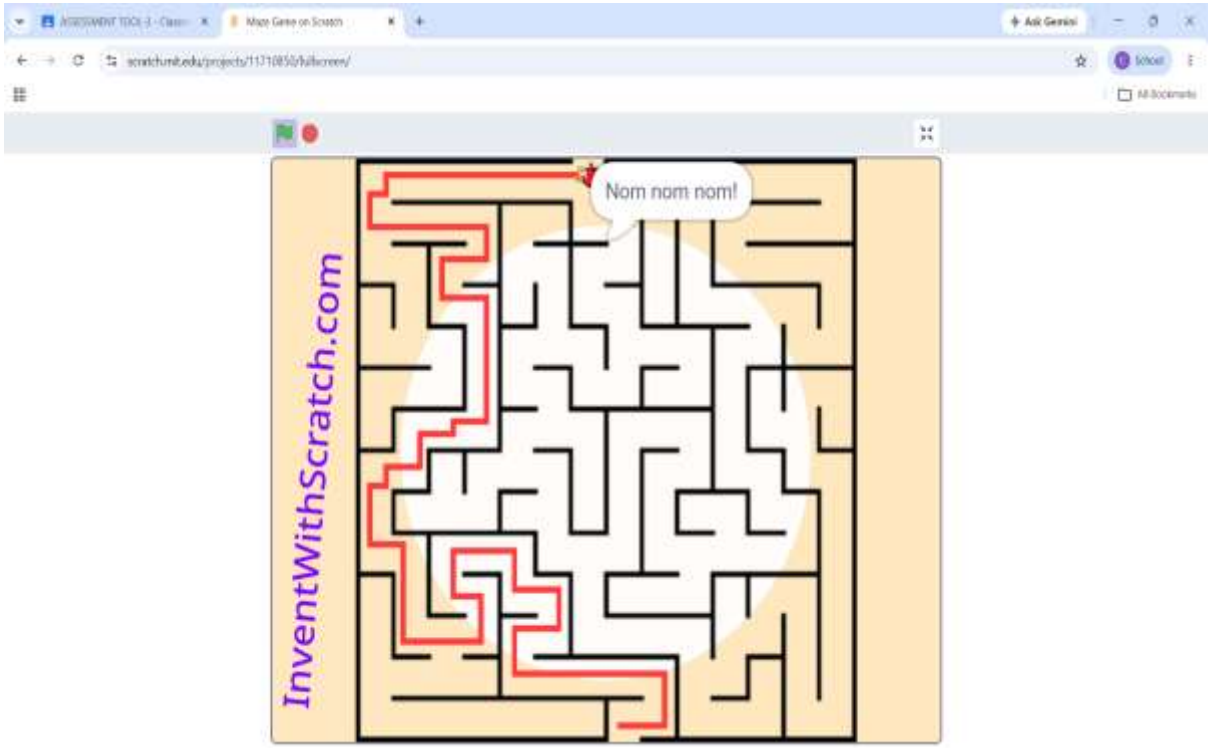
- The maze consists of several walls and narrow passages.
- The player successfully navigated from the starting point to the goal.
- The red line in the maze represents the path followed to reach the destination.
- The maze was completed without crossing any walls.

Result

The AI Maze Escape problem was successfully completed by finding a valid path from the START position to the GOAL position using the Breadth-First Search (BFS) concept. The shortest path was obtained successfully.

Conclusion

The Maze Escape experiment demonstrates how AI search algorithms can efficiently solve pathfinding problems. Breadth-First Search explores the maze systematically and is capable of finding the shortest path between the start and goal positions.



Experiment 2: N-Queens Challenge

Title

N-Queens Challenge – Placing Queens Without Conflicts

Aim

To place all queens on a chessboard such that no two queens attack each other using the Backtracking algorithm.

Problem Statement

The objective of the N-Queens problem is to place **N queens** on an **N × N** chessboard so that no two queens share the same row, column, or diagonal. The goal is to find a valid arrangement with zero conflicts.

Algorithm Used

Backtracking Algorithm

Theory

The Backtracking algorithm is a systematic search technique used to solve constraint satisfaction problems. It places one queen at a time and checks whether the placement is safe. If a conflict occurs, the algorithm removes the last queen and tries another position until a valid solution is found.

Implementation

1. Open the N-Queens game.
2. Select the board size (12 × 12 as specified in the assignment).
3. Place one queen in a safe position on the first row.
4. Continue placing queens row by row while ensuring that no two queens attack each other.
5. If a conflict occurs, remove the queen and try another position.
6. Repeat the process until all queens are placed successfully.

Observation

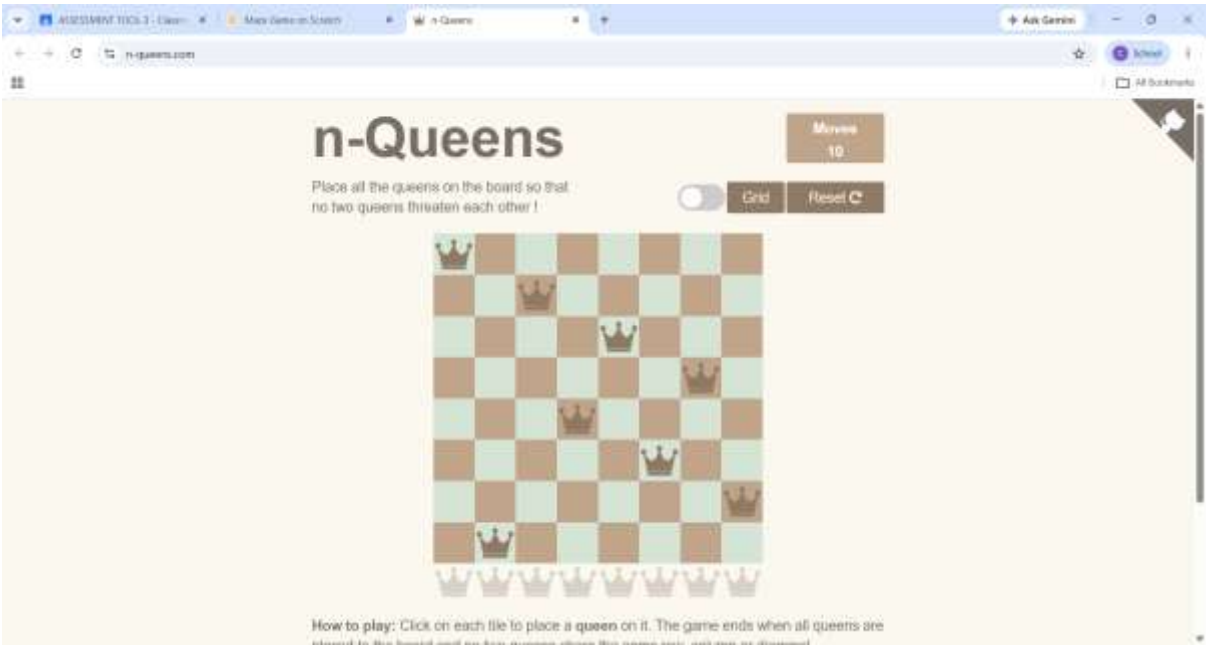
- Queens were placed one by one on the chessboard.
- Each placement was checked to ensure there were no row, column, or diagonal conflicts.
- A valid arrangement was obtained with no attacking queens.
- The puzzle was completed successfully.

Result

The N-Queens problem was successfully solved using the Backtracking algorithm by placing all queens on the chessboard without any conflicts.

Conclusion

The N-Queens problem demonstrates how Backtracking efficiently solves constraint satisfaction problems by exploring possible solutions and eliminating invalid placements until a correct arrangement is found.



Experiment 3: Water Jug Puzzle

Title

Water Jug Puzzle – Measuring Exactly 8 Litres

Aim

To obtain exactly **8 litres** of water using two jugs with capacities of **11 litres** and **9 litres** by applying AI state-space search techniques.

Problem Statement

Two water jugs with capacities of **11 litres** and **9 litres** are provided without any measurement markings. The objective is to measure exactly **8 litres** of water using only the following operations:

- Fill a jug completely.
- Empty a jug.
- Pour water from one jug to another until one becomes empty or the other becomes full.

Algorithm Used

State Space Search (Breadth-First Search / State Space Representation)

Theory

The Water Jug Problem is a classic Artificial Intelligence problem used to demonstrate **State Space Search**. Each possible amount of water in the jugs represents a state, and the allowed operations (fill, empty, and pour) generate new states. The objective is to reach the goal state where exactly **8 litres** of water are present in one of the jugs.

Implementation

1. Open the Water Jug Puzzle game.
2. Use the **11-litre** and **9-litre** jugs.
3. Perform the allowed operations (Fill, Empty, and Pour).
4. Continue changing the water levels until the target quantity of **8 litres** is obtained.
5. Verify that the goal state has been reached.

Observation

- The puzzle was solved by performing a sequence of fill, empty, and transfer operations.
- Different intermediate states were generated before reaching the target state.
- The AI search technique explores possible states to reach the required quantity.

Result

The Water Jug Puzzle was completed successfully by applying the State Space Search approach to obtain the required quantity of water.

Conclusion

The Water Jug Puzzle demonstrates the application of AI search techniques for solving state-space problems. It helps in understanding how intelligent agents explore possible states and identify a sequence of valid actions to reach the desired goal.

Experiment 4: Connect Four AI Challenge

Title

Connect Four AI Challenge

Aim

To play Connect Four against an AI opponent and connect four discs in a row before the opponent.

Problem Statement

Connect Four is a two-player strategy game in which players alternately drop colored discs into a vertical grid. The objective is to connect four discs horizontally, vertically, or diagonally before the opponent.

Algorithm Used

Minimax Algorithm with Alpha-Beta Pruning

Theory

The Minimax algorithm is an Artificial Intelligence search algorithm used in two-player games. It evaluates all possible moves and selects the best move while assuming that the opponent also plays optimally. Alpha-Beta Pruning improves efficiency by eliminating branches that cannot produce a better result.

Implementation

1. Open the Connect Four game.
2. Start a new game against the AI.
3. Drop one disc into a column during each turn.
4. Plan the moves strategically to create a sequence of four connected discs.
5. Block the opponent whenever it attempts to form four connected discs.
6. Continue playing until one player wins.

Observation

- The game was played against the computer.
- The **Blue** player successfully connected **four discs vertically** in the first column.
- The game ended with the message **"Blue wins!"**
- The objective of connecting four consecutive discs was achieved.

Result

The Connect Four AI Challenge was completed successfully. The Blue player won the game by connecting four discs vertically before the opponent.

Conclusion

The Connect Four game demonstrates the application of AI game-search algorithms such as Minimax with Alpha-Beta Pruning. It enhances strategic thinking, decision-making, and planning while competing against an intelligent opponent.

Experiment 5: 8-Puzzle AI Challenge

Title

8-Puzzle AI Challenge

Aim

To solve the 8-Puzzle problem by arranging the numbered tiles in the correct order using the minimum number of moves.

Problem Statement

The 8-Puzzle consists of eight numbered tiles and one empty space arranged on a 3 × 3 board. The objective is to move one tile at a time into the empty space until all the tiles are arranged in ascending numerical order.

Algorithm Used

A* (A-Star) Search Algorithm

Theory

The A* Search Algorithm is a heuristic search algorithm widely used for solving pathfinding and puzzle problems. It evaluates each possible move based on the actual cost from the start state and an estimated cost to reach the goal state. This helps in finding the optimal solution with fewer moves.

Implementation

1. Open the 8-Puzzle game.
2. Shuffle the tiles to generate an initial configuration.
3. Move only the tiles adjacent to the empty space.
4. Continue arranging the tiles in ascending order.
5. Stop when the goal state is achieved.

Goal State

1 2 3

4 5 6

7 8 □

Observation

- The puzzle started in a shuffled configuration.
- The tiles were moved one at a time into the empty space.
- The puzzle reached the goal state with all tiles arranged in ascending order.

- The empty space was positioned at the bottom-right corner.

Result

The 8-Puzzle problem was successfully solved using the A* Search Algorithm by arranging all the numbered tiles in the correct order.

Conclusion

The 8-Puzzle demonstrates the application of heuristic search techniques in Artificial Intelligence. The A* algorithm efficiently identifies the optimal sequence of moves required to reach the goal state, making it one of the most effective algorithms for solving puzzle-based problems.